

國立中山大學
電機工程學系

114-2 實用數位系統設計

HW1 Combinational 8-bit adder/subtractor

Name: Chung EnPing

Student ID: B133015012

Table of Contents

I.Design Process

II.Design Comparison

III.Testbench Design

IV.Waveform Analysis

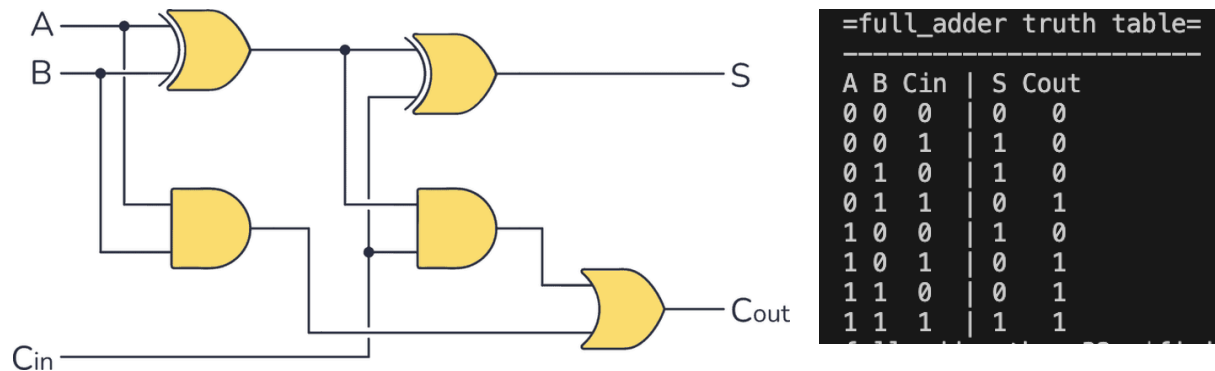
V.Experience and Reflection

VI.Bonus Design

VII.Additional Notes

I.Design Process

We start by making a full adder :



A full adder is a combinational logic circuit used to add three binary inputs: two operand bits A and B, along with a carry-in C_{in}. It produces two outputs: the sum S and the carry-out C_{out}. The sum represents the least significant bit of the result and is calculated using the XOR operation, while the carry-out indicates whether a carry is generated to the next higher bit. The logic expressions for a full adder are $S = A \oplus B \oplus C_{in}$ and $C_{out} = AB + B \cdot C_{in} + A \cdot C_{in}$. In multi-bit adders, the C_{out} of one stage is connected to the C_{in} of the next stage, allowing carries to propagate through the circuit.

Circuit Analysis of Full Adder

Gate Count

From this implementation, the gate usage is:

XOR gates: 2

AND gates: 2

OR gates: 1

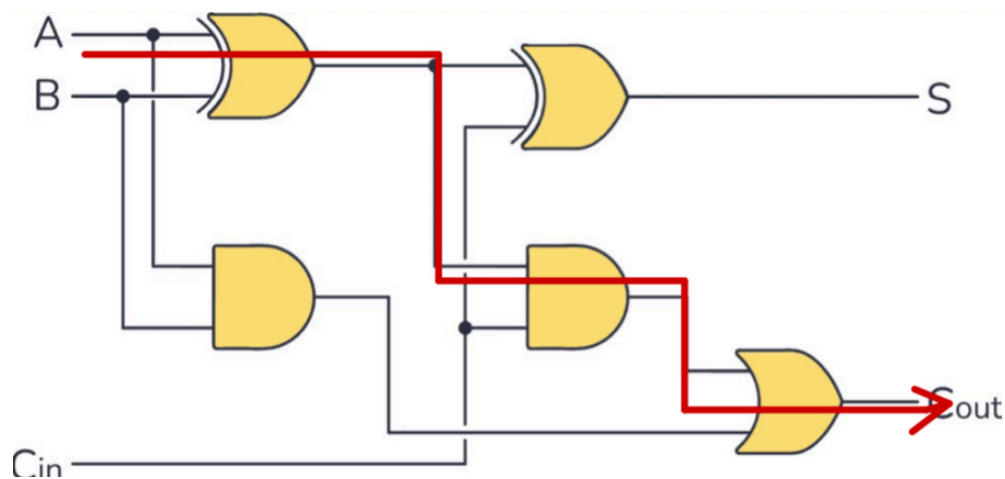
NOT gates: 0

Therefore, the total number of gates in one full adder of my design is 5.

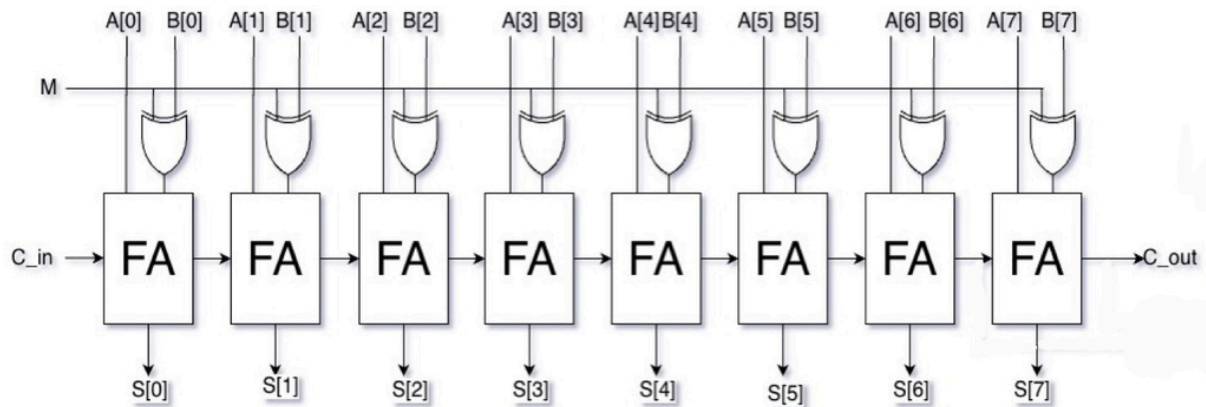
Longest Critical Path

For timing analysis, assuming each gate has one unit delay, the output S is produced through two cascaded XOR gates. The longest path to S is from A or B through axb and then through the second XOR to S, giving a depth of 2 gate levels.

The output C_{out} is produced by “OR-ing” two carry terms. The longest path to C_{out} is from A or B through axb, then through the AND gate that forms cin_axb, and finally through the OR gate. This gives a depth of 3 gate levels. Other paths to Cout are shorter.



Design 0



Design 0 implements an 8-bit adder/subtractor using a chain of full adders (FA) connected in a ripple-carry configuration.

1. Mode Select Using XOR (MUX Behavior)

The circuit incorporates a mode select signal M to allow both addition and subtraction operations within the same hardware. This is achieved by placing an XOR gate before each $B[i]$ input of the full adders.

When $M = 0$, each XOR gate passes $B[i]$ unchanged, and the initial C_{in} is set to 0. Under this condition, the circuit performs standard binary addition:

$$A + B$$

When $M = 1$, each XOR gate inverts $B[i]$, effectively producing $B'[i]$. At the same time, the initial C_{in} is set to 1. This implements the two's complement operation:

$$A + B' + 1$$

which is equivalent to:

$$A - B$$

Thus, the XOR gates function similarly to a multiplexer (MUX), selecting between $B[i]$ and $B'[i]$ depending on the value of M . The combination of XOR logic and controlled C_{in} enables seamless switching between addition and subtraction.

2. Ripple Carry Adder

The circuit is designed as an 8-bit ripple carry adder using eight full adders (FA) in series. Each full adder is responsible for computing one bit of the final sum. For each stage i , the inputs are $A[i]$, $XOR(B[i], M)$, and the carry input C_in , while the outputs are the sum $S[i]$ and carry output C_out .

The defining feature of the ripple carry adder is that the C_out from each full adder is directly connected to the C_in of the next higher-order full adder. This creates a sequential propagation of the carry signal from the least significant bit (LSB) to the most significant bit (MSB). The first stage receives an initial C_in , and each subsequent stage must wait for the previous stage's C_out before producing its result. This results in a simple and modular design, but introduces propagation delay proportional to the number of bits, as the carry must ripple through all eight stages.

Circuit Analysis of Design 0

Gate Count

The circuit is composed of three parts: mode selection ($B \text{ xor } M$ for each bit), and a 8-stage ripple-carry chain of full_adder modules.

At top level, there are 8 XOR gates for mode selection. Each full_adder contains 2 XOR gates, 2 AND gates, and 1 OR gate. Since there are 8 full adders, this contributes 16 XOR, 16 AND, and 8 OR gates.

Therefore, the total hierarchical gate count of the full adder/subtractor is:

XOR = 24

AND = 16,

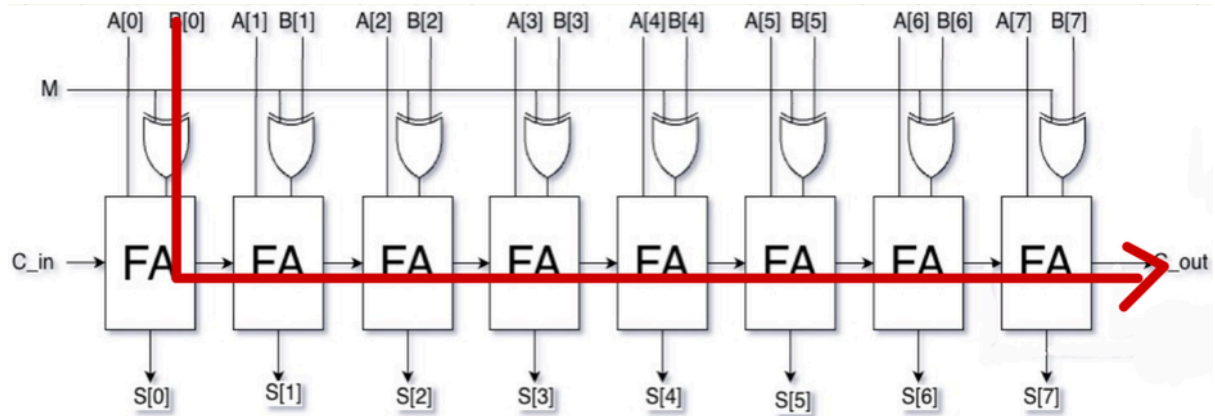
OR = 8

NOT = 0.

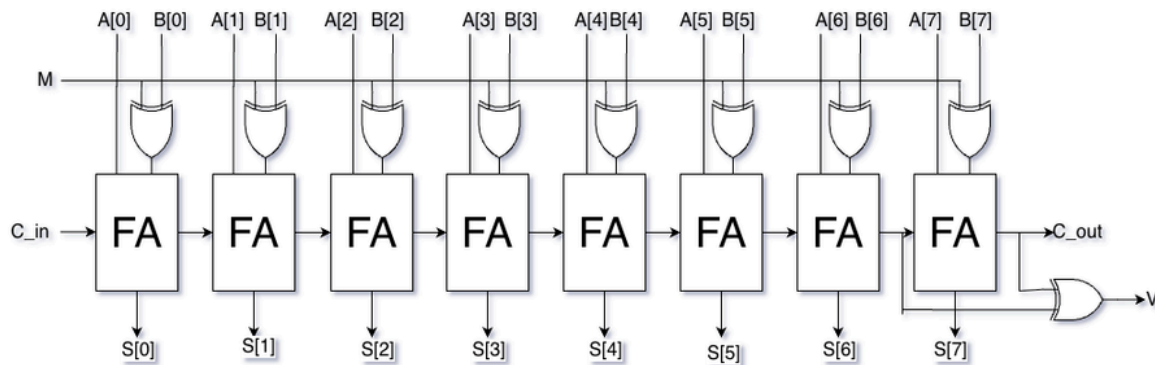
Longest Critical Path

For timing, the longest delay comes from carry propagation through the ripple chain. Under a unit-gate-delay model, the worst path starts from low-order control/data influence (such as M through $XOR[0]$), propagates through all carry

stages to C_{out} . The estimated longest critical path is about 18 gate levels to C_{out} .



Design I



Circuit Analysis of Design I

Gate Count

The circuit is composed of three parts: mode selection ($B \text{ xor } M$ for each bit), an 8-stage ripple-carry chain of full_adder modules, and overflow detection ($V = C[6] \text{ xor } C_{\text{out}}$). At top level, there are 8 XOR gates for mode selection and 1 XOR gate for overflow detection. Each full_adder contains 2 XOR gates, 2 AND gates, and 1 OR gate. Since there are 8 full adders, this contributes 16 XOR, 16 AND, and 8 OR gates.

Therefore, the total hierarchical gate count of the full adder/subtractor is:

XOR = 25

AND = 16,

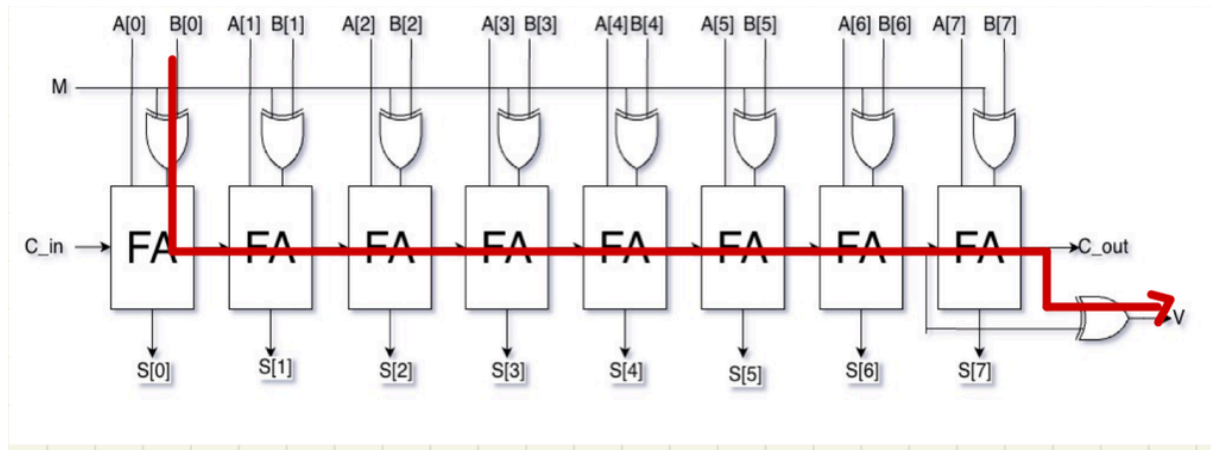
OR = 8

NOT = 0.

Longest Critical Path

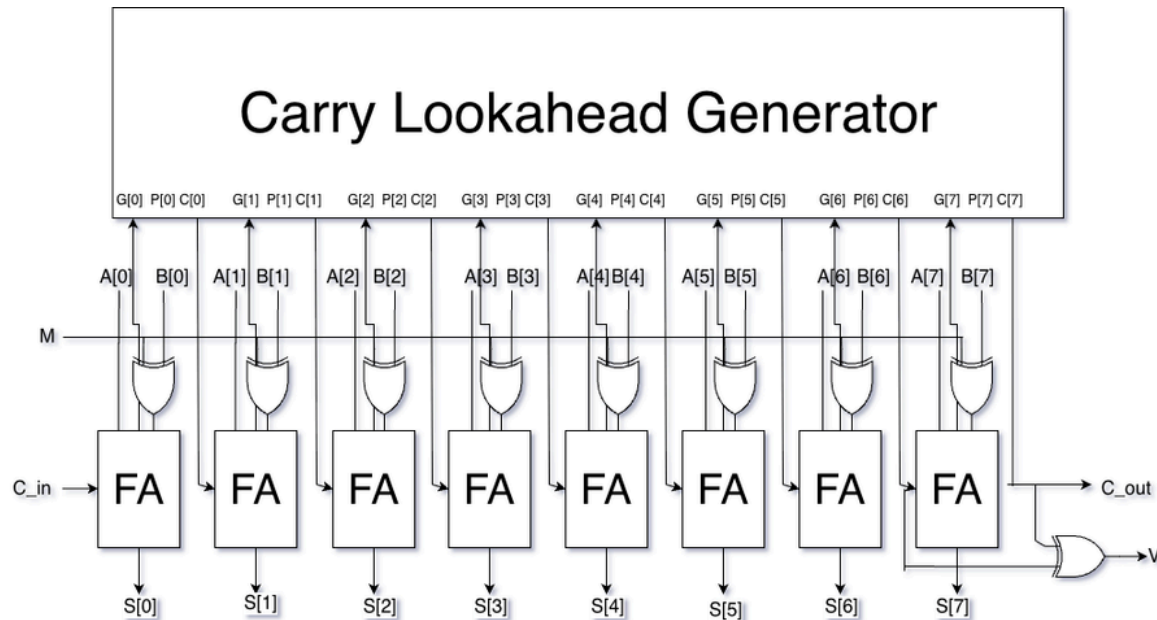
For timing, the longest delay comes from carry propagation through the ripple chain. Under a unit-gate-delay model, the worst path starts from low-order control/data influence (such as M through XOR[0]), propagates through all carry stages to C_{out} , and then goes through the overflow XOR to V . The estimated longest critical path is about 19 gate levels to V (about 18 gate levels to C_{out}).

Thus, the overall critical output is overflow V , and the bottleneck is ripple-carry propagation across 8 bits.



Design II

We realize that with the ripple adder design, the worst delay needs to pass through 18 gates(overflow detection bit V). So we design a carry lookahead generator to create the C_in inputs while the adders are working.



1. Improvement over Ripple Carry Adder

In this design, the conventional ripple carry structure is enhanced by incorporating a **carry lookahead generator (CLG)**. While the circuit still uses eight full adders (FA) to compute the sum bits S[0] to S[7], the key improvement lies in how the carry signals are generated.

In a ripple carry adder, each stage must wait for the previous stage's C_out before producing its own result. This creates a cumulative delay as the carry propagates sequentially from the least significant bit (LSB) to the most significant bit (MSB).

In contrast, the carry lookahead generator computes all carry signals in advance using the input signals. Instead of waiting for carry propagation, each full adder receives its corresponding C_in directly from the CLG. This significantly reduces the overall propagation delay and improves the speed of the circuit, especially for higher bit-width operations.

2. Logic of the Carry Lookahead Generator

The carry lookahead generator operates using two key signals for each bit position i :

- **Generate:**
- $G[i] = A[i] B[i]$
- This indicates that a carry will definitely be generated at bit i , regardless of C_{in} .
- **Propagate:**
- $P[i] = \text{XOR}(A[i], B[i])$
- This indicates that a carry will propagate through bit i if a carry is present at C_{in} .

Using these signals, the carry outputs for each stage can be expressed without waiting for ripple effects:

$$C[1] = G[0] + P[0] C_{in}$$

$$C[2] = G[1] + P[1]G[0] + P[1]P[0]C_{in}$$

$$C[3] = G[2] + P[2]G[1] + P[2]P[1]G[0] + P[2]P[1]P[0]C_{in}$$

This pattern continues up to:

$$C[8] = C_{out}$$

Each carry $C[i]$ is computed as a sum of products involving generate and propagate terms. This allows all carries to be determined in parallel, rather than sequentially.

3. Integration with Adder–Subtractor Functionality

Similar to the previous design, the circuit supports both addition and subtraction using the mode signal M . Each $B[i]$ input passes through an XOR gate controlled by M :

- When $M = 0$: $B[i]$ remains unchanged, and $C_{in} = 0$, resulting in addition.
- When $M = 1$: $B[i]$ is inverted, and $C_{in} = 1$, implementing two's complement subtraction.

The propagate signal $P[i] = A[i]B[i]$ is naturally compatible with this structure, since the XOR operation is already used in both the carry logic and sum generation.

4. Overflow Detection

Overflow detection is handled in the same way as in the ripple carry design. The overflow signal V is determined by comparing the carry into and carry out of the most significant bit.

$$V = C_{in}C_{out}$$

Here, C_{in} refers to the carry entering the MSB stage, and C_{out} is the final carry output of the circuit. A mismatch between these two signals indicates overflow in signed arithmetic.

Circuit Analysis of CLA Generator

The module is implemented directly as sum-of-products carry equations from $C[0]$ to $C[7]$. Each carry output has one OR gate at the final stage and multiple product terms generated by AND gates. The number of AND terms grows by bit position: 1, 2, 3, 4, 5, 6, 7, 8.

From the source, the total primitive gates are:

$$\text{XOR} = 0$$

$$\text{AND} = 36$$

$$\text{OR} = 8$$

$$\text{NOT} = 0$$

So the total primitive gate count is 44, with no inverter usage.

Circuit Analysis of Design 2

Gate Count

Top-level primitive gates in `eight_bit_adder_subtractor` are 8 mode-select XOR gates plus 1 overflow XOR gate, giving 9 XOR gates. Each `full_adder` uses 2 XOR, 2 AND, and 1 OR gate. With 8 instances, this contributes 16 XOR, 16 AND, and 8 OR gates. The CLA block uses 36 AND gates and 8 OR gates. Therefore, the full hierarchical gate count of the Version 2 adder/subtractor is:

XOR = 25

AND = 52

OR = 16

NOT = 0

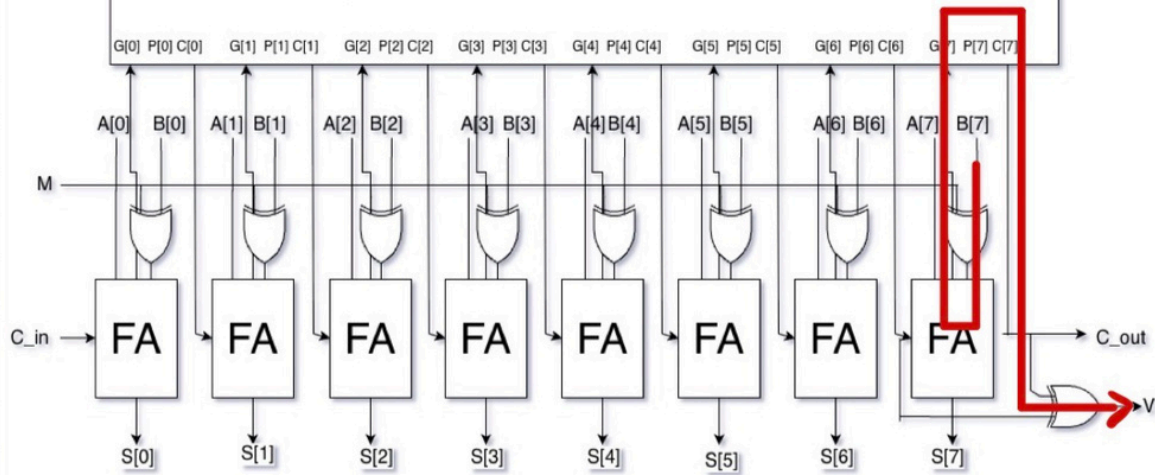
Longest Critical Path

Assuming a unit-delay model where each primitive xor/and/or gate has one delay and each multi-input primitive is counted as one gate level, the longest path is through the carry-lookahead-dependent outputs (`SUM[7]` or `V`), not through ripple carry.

A worst-case path is from `M/B` into mode-select XOR, then into `P/G` generation inside a full adder, then through CLA term generation (AND) and carry combine (OR), and finally through the last XOR stage (`SUM[7]` XOR or overflow XOR). This gives an estimated maximum depth of about 5 gate levels.

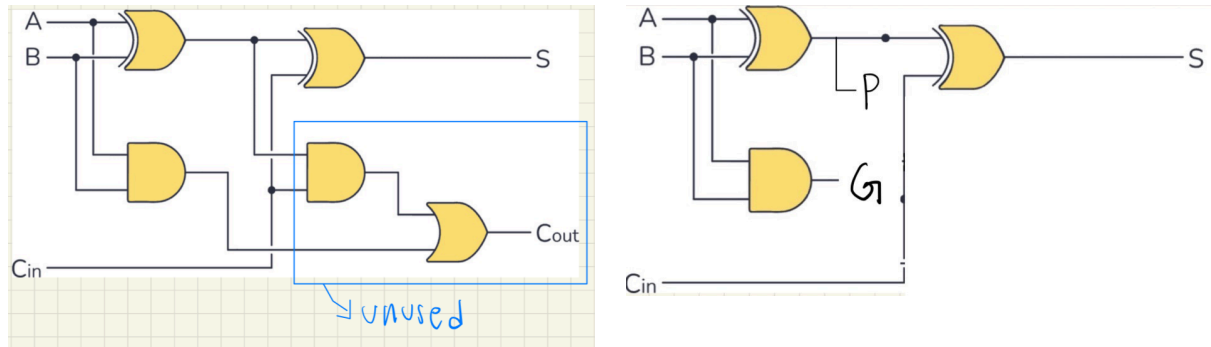
So, for Version 2, the longest critical path is approximately 5 gate levels, and the critical outputs are the top-bit sum/overflow paths that depend on CLA carries.

Carry Lookahead Generator



Design III (Final Version)

After implementing the CLA generator, we see that the C_{out} bits of each full adder are no longer used. We can save two logic gates for each adder by removing the C_{out} bit.



This creates a new adder design which I have named “No_Cout_Adder”.

Circuit Analysis of Design no_Cout_Adder

The no_Cout_adder is a reduced full-adder cell designed for carry-lookahead architectures. It still computes the sum bit, but it does not compute a local carry-out output. Inside the module, A and B are first XORed to form axb . The sum output is then $S = axb \text{ xor } Cin$, which is the standard full-adder sum equation. The module also computes $G = A \text{ and } B$ and sets $P = axb$. These are the generate and propagate signals used by carry-lookahead logic. It is called no_Cout_adder because the normal carry-out equations are intentionally removed. In a regular full adder, carry-out would be $Cout = G + P * Cin$. Here, that part is omitted so that carry is not produced locally. Instead, all bit positions send P and G to the separate carry-lookahead generator, which computes all carries in parallel. So the module's purpose is to provide local sum and P/G information, while delegating carry computation to the CLA block for faster overall timing.

Gate Count

XOR gates: 2

AND gates: 1

OR gates: 0

NOT gates: 0

Circuit Analysis of Design 3

Gate Count

At top level, there are 8 XOR gates for mode selection ($B \text{ xor } M$) and 1 XOR gate for overflow detection ($V = C[6] \text{ xor } C_{\text{out}}$), so top-level primitives contribute 9 XOR gates. Each `no_Cout_adder` contains 2 XOR gates and 1 AND gate, with no OR and no NOT. With 8 instances, this contributes 16 XOR and 8 AND. The CLA generator contributes 36 AND gates and 8 OR gates.

Combining all hierarchy levels, the total primitive count is:

XOR = 25

AND = 44

OR = 8

NOT = 0

Longest Critical Path

Under a unit-delay primitive model (each xor, and, or counted as one gate level, including multi-input forms), the worst path is through CLA-dependent outputs. A representative worst-case path is from M/B through mode-select XOR, through `no_Cout_adder` P-generation XOR, through CLA product-term AND and carry-combine OR, and then through the destination XOR ($SUM[7]$ or overflow XOR V).

This gives an estimated maximum depth of about 5 gate levels, the same as Design 2.

So the longest critical path of the Design 3 adder/subtractor is approximately 5 gate levels, with critical endpoints at $SUM[7]$ and V (both carry-dependent).

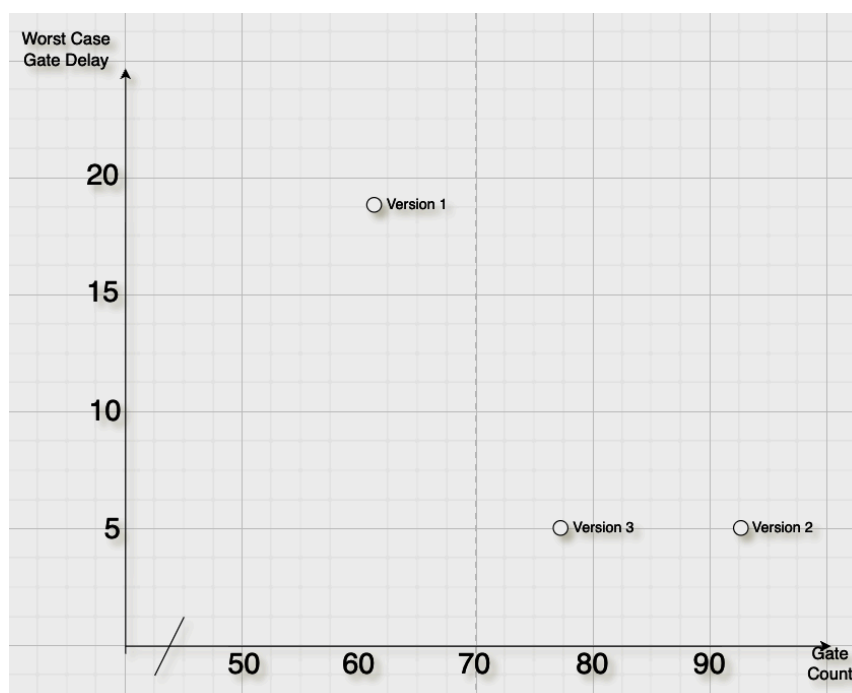
II.Design Comparison

Version 1 and Version 3 implement the same 8-bit adder/subtractor function, but they use different carry strategies and therefore have different performance and hardware tradeoffs.

Version 1 is a ripple-carry design. It uses full adders connected in series, so each bit must wait for the previous carry. This makes the structure simple and gate usage lower, but delay grows with bit position and the worst-case path is long. In your earlier analysis model, the critical path is about 19 gate levels to overflow output V.

Version 3 is a carry-lookahead-based design. It still computes per-bit sum, but it separates local P/G generation (no_Cout_adder) from carry generation (eight_bit_carry_lookahead_generator). Carries are computed in parallel with lookahead equations instead of rippling through all bits. This significantly reduces worst-case delay; in your unit-delay model, the critical path is about 5 gate levels.

In gate count terms, Version 3 uses more combinational carry logic than Version 1. Version 1 total is XOR 25, AND 16, OR 8, NOT 0, while Version 3 total is XOR 25, AND 44, OR 8, NOT 0. So Version 3 increases area/complexity (mainly AND count) to gain speed.



For verification style, both can be tested functionally the same way, but Version 3 especially benefits from waveform-oriented benches that highlight carry behavior under overflow/borrow edge cases. Practically, Version 1 is better for simpler implementation and teaching ripple-carry behavior, while Version 3 is better when timing performance matters.

As you can see from the graph the trade off you make for speed is the increase number of gates.

III. Testbench Design

Ideally, we want to design focused verification bench, not an exhaustive one. The goal is to make waveform reading clean while still proving core behavior. The process starts by instantiating the UUT (eight_bit_adder_subtractor) and building a reusable expected-value model (calculate_expected_value) as a task so every test can be checked automatically.

Then a reusable checking task (run_case) is used. Each case sets A/B/M, waits for settle time, computes expected SUM/C_out/V, compares UUT output vs expected, and prints PASS/FAIL. This keeps verification consistent and avoids manual checking.

The chosen vectors are “most relevant” vectors: normal addition/subtraction, carry-out, borrow, signed overflow in both directions, identity cases (A+0, A-0, A-A), and alternating-bit patterns (0x55/0xAA). Those directly target functionality plus corner/edge cases.

Waveform generation is enabled with \$dumpfile("testbench5.vcd") and \$dumpvars(0, testbench5). Each case is held for a fixed time window (10 ns) so transitions are separated in time, making GTKWave/nWave inspection clear.

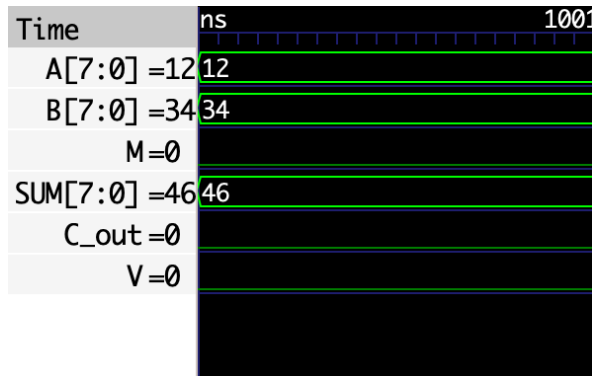
Terminal Output:

```
=== testbench5: relevant functional + edge-case vectors ===
Each case is held for 10ns for clear waveform time frames.


| time(ns) | M | A  | B  | DUT_SUM | DUT_C | DUT_V | EXP_SUM | EXP_C | EXP_V | RESULT | Description                        |
|----------|---|----|----|---------|-------|-------|---------|-------|-------|--------|------------------------------------|
| 1000     | 0 | 12 | 34 | 46      | 0     | 0     | 46      | 0     | 0     | PASS   | ADD basic: 0x12 + 0x34             |
| 12000    | 0 | ff | 01 | 00      | 1     | 0     | 00      | 1     | 0     | PASS   | ADD carry-out: 0xFF + 0x01         |
| 23000    | 0 | 7f | 01 | 80      | 0     | 1     | 80      | 0     | 1     | PASS   | ADD signed overflow: 0x7F + 0x01   |
| 34000    | 0 | 80 | 80 | 00      | 1     | 1     | 00      | 1     | 1     | PASS   | ADD negative overflow: 0x80 + 0x80 |
| 45000    | 1 | 3a | 19 | 21      | 1     | 0     | 21      | 1     | 0     | PASS   | SUB basic: 0x3A - 0x19             |
| 56000    | 1 | 05 | 03 | 02      | 1     | 0     | 02      | 1     | 0     | PASS   | SUB no borrow: 0x05 - 0x03         |
| 67000    | 1 | 00 | 01 | ff      | 0     | 0     | ff      | 0     | 0     | PASS   | SUB borrow: 0x00 - 0x01            |
| 78000    | 1 | 80 | 01 | 7f      | 1     | 1     | 7f      | 1     | 1     | PASS   | SUB signed overflow: 0x80 - 0x01   |
| 89000    | 1 | 7f | ff | 80      | 0     | 1     | 80      | 0     | 1     | PASS   | SUB signed overflow: 0x7F - 0xFF   |
| 100000   | 0 | a5 | 00 | a5      | 0     | 0     | a5      | 0     | 0     | PASS   | ADD identity: A + 0                |
| 111000   | 1 | a5 | 00 | a5      | 1     | 0     | a5      | 1     | 0     | PASS   | SUB identity: A - 0                |
| 122000   | 1 | 5c | 5c | 00      | 1     | 0     | 00      | 1     | 0     | PASS   | SUB self: A - A                    |
| 133000   | 0 | 55 | aa | ff      | 0     | 0     | ff      | 0     | 0     | PASS   | ADD alternating bits: 0x55 + 0xAA  |
| 144000   | 1 | aa | 55 | 55      | 1     | 1     | 55      | 1     | 1     | PASS   | SUB alternating bits: 0xAA - 0x55  |

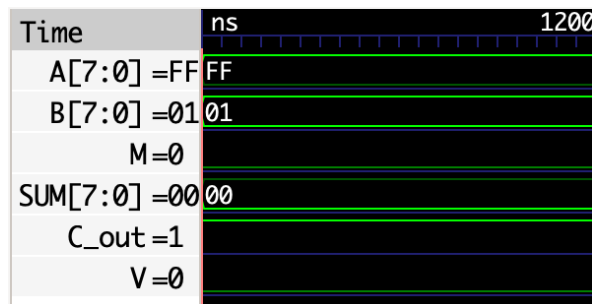

=== SUMMARY ===
Total checks: 14
Errors: 0
RESULT: PASS
```

IV. Waveform Analysis



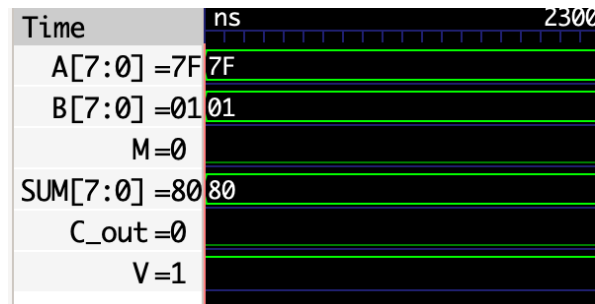
A=12 B=34 M=0 | Sum=46 C_out=0 V=0 |

ADD basic: 0x12 + 0x34



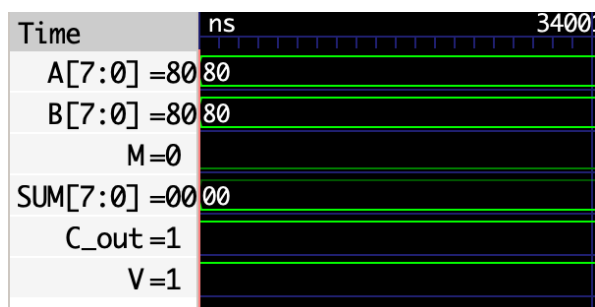
A=FF B=01 M=0 | Sum=00 C_out=1 V=0 |

ADD carry-out: 0xFF + 0x01



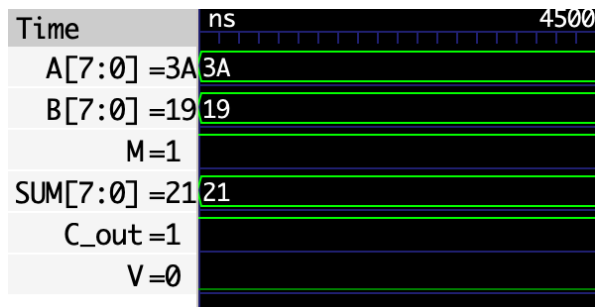
A=7F B=01 M=0 | Sum=80 C_out=0 V=1 |

ADD signed overflow: 0x7F + 0x01



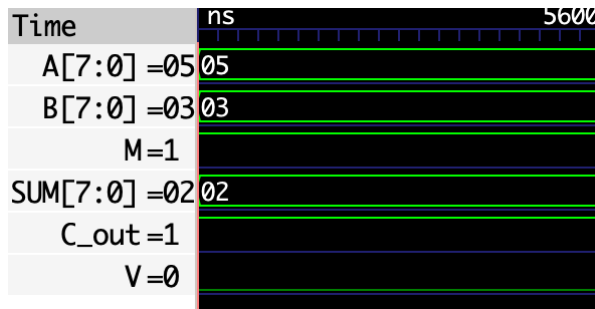
A=80 B=80 M=0 | Sum=00 C_out=1 V=1 |

ADD negative overflow: 0x80 + 0x80



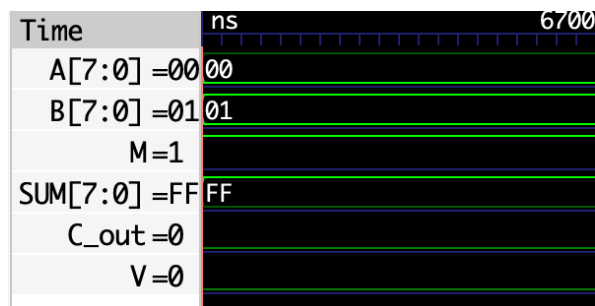
A=3A B=19 M=1 | Sum=21 C_out=1 V=0 |

SUB basic: 0x3A - 0x19



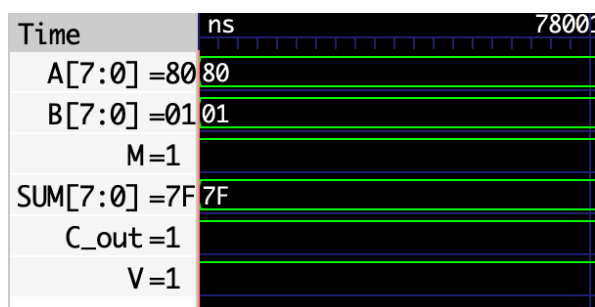
A=05 B=03 M=1 | Sum=02 C_out=1 V=0 |

SUB no borrow: 0x05 - 0x03



A=00 B=01 M=1 | Sum=FF C_out=0 V=0 |

SUB borrow: 0x00 - 0x01



A=80 B=01 M=1 | Sum=7F C_out=1 V=1 |

SUB signed overflow: 0x80 - 0x01

Time	ns	8900
A[7:0] =7F	7F	
B[7:0] =FF	FF	
M=1		
SUM[7:0] =80	80	
C_out =0		
V=1		

A=7F B=FF M=1 | Sum=80 C_out=0 V=1 |

SUB signed overflow: 0x7F - 0xFF

Time	ns	10000
A[7:0] =A5	A5	
B[7:0] =00	00	
M=1		
SUM[7:0] =A5	A5	
C_out =1		
V=0		

A=A5 B=00 M=0 | Sum=A5 C_out=0 V=0 |

ADD identity: A + 0

Time	ns	11100
A[7:0] =A5	A5	
B[7:0] =00	00	
M=1		
SUM[7:0] =A5	A5	
C_out =1		
V=0		

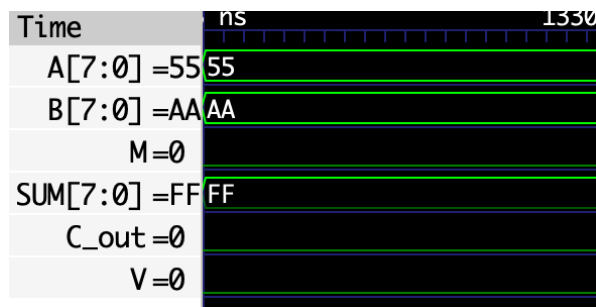
A=A5 B=00 M=1 | Sum=A5 C_out=1 V=0 |

SUB identity: A - 0

Time	ns	12200
A[7:0] =5C	5C	
B[7:0] =5C	5C	
M=1		
SUM[7:0] =00	00	
C_out =1		
V=0		

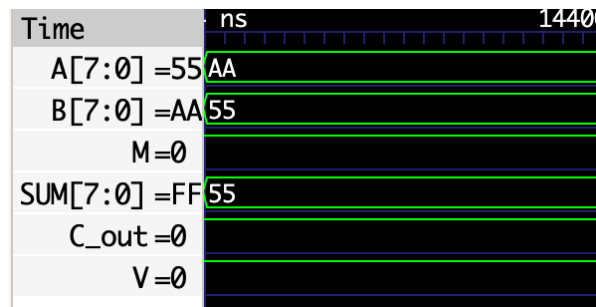
A=5C B=5C M=1 | Sum=00 C_out=1 V=0 |

SUB self: A - A



A=55 B=AA M=0 | Sum=FF C_out=0 V=0 |

ADD alternating bits: 0x55 + 0xAA



A=AA B=55 M=1 | Sum=55 C_out=1 V=1 |

SUB alternating bits: 0xAA - 0x55

V. Experience and Reflection

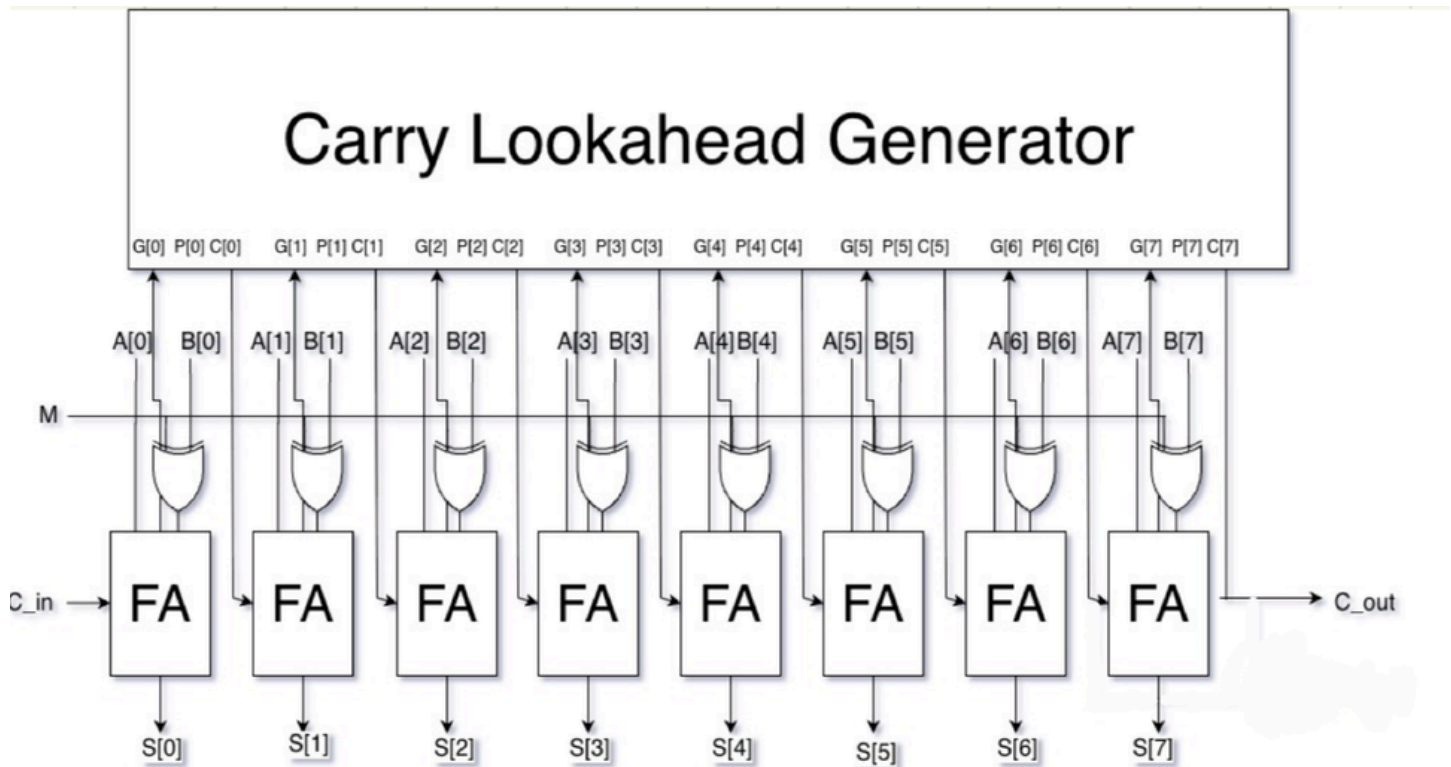
Building the adder/subtractor across three versions felt like a progression from “make it work” to “make it better.” Version 1 was the most direct implementation. I used a standard full adder and chained eight stages as ripple carry, then added mode control with $B \oplus M$ and overflow detection. The main value of this stage was clarity. It was easy to understand, easy to debug, and good for confirming the arithmetic logic. The limitation became obvious quickly: carry propagation dominated delay, so timing was the weak point.

In Version 2, I shifted focus from function-only design to performance-aware structure. I kept the same arithmetic behavior, but changed the architecture to carry lookahead. That required generating propagate and generate signals in each bit cell and building a separate carry-lookahead generator. This version taught me that improving speed usually increases logic complexity. I had to think more carefully about equations, signal naming, and module interfaces because more wires and intermediate terms were involved. Verification also became more important, since a small carry-equation mistake can silently break many cases.

Version 3 was a refinement step. I separated responsibilities more clearly by using `no_Cout_adder` cells to produce only S, P, and G, and let the CLA block handle all carry outputs. This made the structure cleaner and more modular than Version 2. It also reinforced a design principle I learned during the project: when modules have a single clear purpose, both analysis and testing become easier. I spent more time building better testbenches in this stage, including edge-case-focused vectors, automated expected-result comparison, readable terminal formatting, and waveform dumping for timing/transition inspection.

Reflecting on the full process, I thought it was going to be easy since I was familiar with Coding and Linux commands in general. But the design process was not as easy as just writing a code to do a basic function. It had to be broken down into steps, and making different types of improvements through the different versions.

Bonus Design- CLA_Unsigned_without Overflow



CLA unsigned-without-overflow and CLA signed-with-overflow circuits are the same data path for sum and c_out . The only difference is that signed version adds one extra output, overflow, and one extra gate.

Circuit Analysis

Gate Count

Combining all hierarchy levels, the total primitive count is:

XOR = 24

AND = 44

OR = 8

NOT = 0

Longest Critical Path

The longest path to $sum[7]$ is about 4 gate delays, and the longest path to C_out is about 3 gate delays in this model. In the signed version, overflow is formed by one more XOR from $C[6]$ and c_out , so worst-case to overflow is also about 4 delays. So functionally, signed-with-overflow is the same CLA core plus one extra XOR stage for the overflow flag.

Testbench(Different one) Terminal Output

```

===== testbench: relevant functional + edge-case vectors =====
| time(ns) | M | A | B | DUT_SUM | DUT_C | EXP_SUM | EXP_C | RESULT | Description |
|-----|---|---|---|-----|-----|-----|-----|-----|-----|
| 20000 | 0 | 12 | 34 | 46 | 0 | 46 | 0 | PASS | ADD basic: 0x12 + 0x34 |
| 48000 | 0 | ff | 01 | 00 | 1 | 00 | 1 | PASS | ADD carry-out: 0xFF + 0x01 |
| 76000 | 0 | ff | ff | fe | 1 | fe | 1 | PASS | ADD max + max: 0xFF + 0xFF |
| 104000 | 0 | 00 | 00 | 00 | 0 | 00 | 0 | PASS | ADD zero + zero |
| 132000 | 1 | 3a | 19 | 21 | 1 | 21 | 1 | PASS | SUB basic: 0x3A - 0x19 |
| 160000 | 1 | 05 | 03 | 02 | 1 | 02 | 1 | PASS | SUB no borrow: 0x05 - 0x03 |
| 188000 | 1 | 00 | 01 | ff | 0 | ff | 0 | PASS | SUB borrow: 0x00 - 0x01 |
| 216000 | 1 | 00 | ff | 01 | 0 | 01 | 0 | PASS | SUB underflow edge: 0x00 - 0xFF |
| 244000 | 1 | a5 | a5 | 00 | 1 | 00 | 1 | PASS | SUB self: A - A |
| 272000 | 0 | a5 | 00 | a5 | 0 | a5 | 0 | PASS | ADD identity: A + 0 |
| 300000 | 1 | a5 | 00 | a5 | 1 | a5 | 1 | PASS | SUB identity: A - 0 |
| 328000 | 0 | 55 | aa | ff | 0 | ff | 0 | PASS | ADD alternating bits: 0x55 + 0xAA |
| 356000 | 1 | aa | 55 | 55 | 1 | 55 | 1 | PASS | SUB alternating bits: 0xAA - 0x55 |
| 384000 | 0 | 0f | 01 | 10 | 0 | 10 | 0 | PASS | ADD nibble carry chain: 0x0F + 0x01 |
===== SUMMARY =====
Total checks: 14
Errors: 0
RESULT: PASS

```

VII. Additional Notes

Design process on iPad:

